	SKT Software Solution	CÓDIGO: MI-DES-GUI-001
SKT SOFTWARE SOLUTION	PROCESO: DESARROLLO DE SOFTWARE	VERSIÓN: 1.0
	Configuración del entorno	ELABORACIÓN: 25/03/2026
	Documento técnico académico	ÚLTIMA REVISIÓN: 25/03/2026

1. Título

Configuración del entorno local de desarrollo con Docker y Git para el Sistema Web Progresivo LUPSI

2. Introducción

El presente documento describe la configuración del entorno local de desarrollo que será utilizado para la construcción del Sistema Web Progresivo (PWA) LUPSI. Su propósito es estandarizar el trabajo técnico del equipo mediante el uso de herramientas de control de versiones y de virtualización de servicios, garantizando que todos los integrantes dispongan de un entorno homogéneo, reproducible y alineado con la arquitectura definida para el proyecto.

La solución LUPSI se desarrolla como una aplicación web progresiva orientada a la gestión y reserva de citas médicas, apoyada en un stack tecnológico compuesto por Angular para el frontend, NestJS para el backend, Supabase para la base de datos y autenticación, y Cloudinary para la gestión de archivos PDF. Para el trabajo local del equipo se emplean Git como sistema de control de versiones y Docker como plataforma de contenedorización.

Este informe se centra exclusivamente en la configuración local de Docker y Git. No aborda despliegue en producción, integración continua ni automatización de entrega, ya que el objetivo es dejar documentado el entorno base para desarrollo, pruebas y validación interna del equipo. La adopción de este entorno permite reducir inconsistencias entre máquinas, facilitar la incorporación de nuevos miembros y mejorar la trazabilidad técnica de los cambios realizados durante el ciclo de vida del proyecto.

3. Objetivo

Definir y documentar la configuración del entorno local de desarrollo del proyecto LUPSI mediante el uso de Git y Docker, estableciendo una base técnica uniforme para el trabajo colaborativo, la administración de versiones del código fuente y la ejecución reproducible de los servicios necesarios para el desarrollo del sistema.

4. Definiciones

- **Entorno local de desarrollo:** conjunto de herramientas, configuraciones y servicios instalados en la computadora del desarrollador para construir, ejecutar y probar el sistema antes de su despliegue.
- **Git:** sistema de control de versiones distribuido que permite registrar cambios, trabajar con ramas, coordinar aportes del equipo y mantener historial trazable del código fuente.
- **Docker:** plataforma de contenerización que permite empaquetar aplicaciones y servicios con sus dependencias para ejecutarlos de forma consistente en diferentes equipos.
- **Contenedor:** instancia aislada y ligera que ejecuta un servicio o aplicación con su propia configuración, sin afectar de forma directa al sistema anfitrión.
- **Imagen Docker:** plantilla inmutable desde la cual se crean contenedores; contiene la aplicación, bibliotecas y parámetros necesarios para su ejecución.
- **Repositorio:** espacio centralizado, local o remoto, donde se almacena el historial del proyecto y se organizan las contribuciones del equipo.
- **Branch o rama:** línea de trabajo independiente dentro del repositorio que permite desarrollar funcionalidades o correcciones sin alterar la versión principal.

5. Alcance del Entorno

La configuración documentada en este informe comprende únicamente el ambiente de trabajo local necesario para el desarrollo del sistema LUPSI. En particular, incluye la instalación y uso de Git, la creación de un flujo básico de ramas, la preparación de contenedores con Docker y la definición de una estructura de trabajo compatible con el stack Angular, NestJS, Supabase y Cloudinary.

Fuera del alcance de este documento quedan los siguientes elementos:

- despliegue en servidores de producción;
- configuración de dominio público o certificados TLS/SSL;

- automatización con pipelines de integración continua;
- monitoreo de infraestructura en la nube;
- administración avanzada de costos o escalamiento de servicios.

6. Requisitos del Entorno Local

Para desarrollar el proyecto de forma ordenada, se recomienda que cada integrante cuente con una estación de trabajo con acceso a internet y con los siguientes componentes instalados:

- Git para control de versiones.
- Docker Desktop o Docker Engine, según el sistema operativo.
- Node.js compatible con Angular y NestJS.
- Un editor de código fuente, preferentemente Visual Studio Code.
- Acceso a una cuenta en GitHub, GitLab o GitHub Classroom para alojar el repositorio remoto.

Además, se sugiere que la estructura del proyecto mantenga separación clara entre frontend, backend y recursos compartidos, con el fin de facilitar la revisión del código y reducir errores de integración.

7. Configuración de Git

Git será el mecanismo principal para el control de versiones del proyecto. Su función será registrar el avance del código, habilitar la colaboración simultánea del equipo y permitir la recuperación de versiones anteriores en caso de errores o regresiones.

7.1. Instalación y verificación

Una vez instalado Git, se debe comprobar su disponibilidad en la terminal con el comando:

```
git --version
```

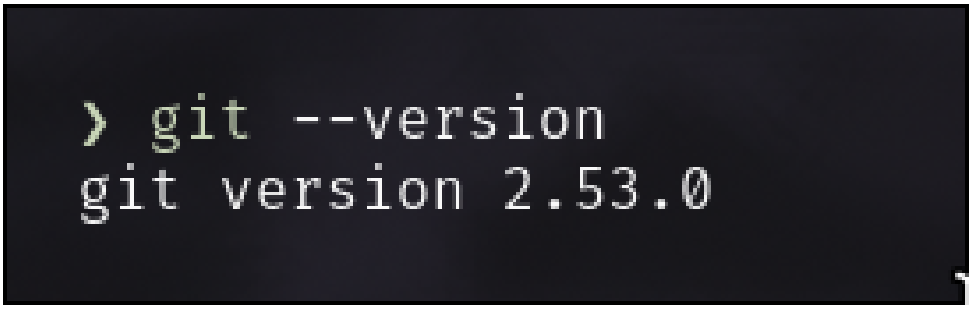


Figura 1: Verificación de la instalación de Git

Posteriormente, cada integrante debe registrar su identidad de usuario para que los commits queden correctamente asignados:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
```

7.2. Estructura del repositorio

Se propone una organización básica del repositorio como la siguiente:

```
/lupsi
  /frontend
  /backend
  /docs
  /docker
  README.md
```

Esta estructura permite separar claramente la aplicación cliente, la lógica del servidor, la documentación técnica y los archivos de soporte para contenedores.

7.3. Flujo de trabajo con ramas

Para evitar conflictos y conservar estabilidad en la rama principal, se recomienda un flujo simple de ramas:

- **main**: versión estable del proyecto.

- **develop**: rama de integración para pruebas internas.
- **feature/*nombre-funcionalidad***: ramas temporales para nuevas funcionalidades.
- **fix/*descripcion***: ramas para corrección de errores.

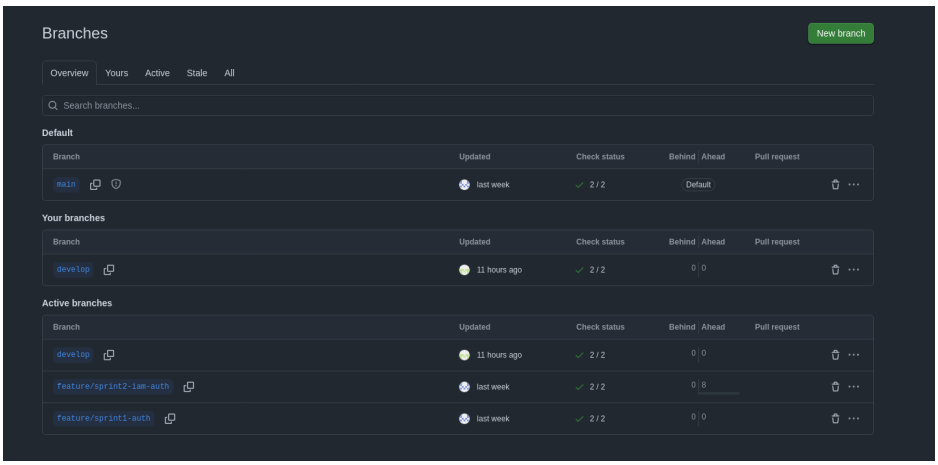


Figura 2: Flujo de trabajo con ramas en Git

Cada cambio debe ingresar al repositorio mediante commits descriptivos y, de ser posible, a través de revisiones por pares antes de fusionarse con la rama principal. Ejemplos de mensajes adecuados son:

```
feat: agregar base del formulario de registro
fix: corregir validación de cédula módulo 10
docs: actualizar instrucciones de instalación
```

7.4. Buenas prácticas de control de versiones

- Realizar commits pequeños y frecuentes.
- Evitar subir archivos temporales, dependencias o credenciales.
- Mantener un historial comprensible y coherente.
- Sincronizar cambios con el repositorio remoto al finalizar cada jornada de trabajo.

- Proteger la rama principal contra modificaciones directas no revisadas.

8. Configuración de Docker

Docker se utilizará para reproducir localmente los servicios que forman parte del stack técnico, minimizando diferencias entre equipos y reduciendo problemas de compatibilidad. En el contexto de LUPSI, Docker es especialmente útil para levantar servicios de backend, entornos auxiliares y dependencias de prueba de manera controlada.

8.1. Propósito en el proyecto

El uso de Docker ofrece las siguientes ventajas:

- homogeneidad entre entornos de desarrollo;
- facilidad para reinstalar el proyecto sin complejidad adicional;
- menor riesgo de errores por diferencias de versiones;
- aislamiento de servicios para pruebas locales;
- apoyo a la documentación técnica del sistema.

8.2. Estructura recomendada de archivos Docker

Se recomienda crear una carpeta dedicada para la configuración de contenedores:

```
/docker
  docker-compose.yml
  /backend
    Dockerfile
  /frontend
    Dockerfile
```

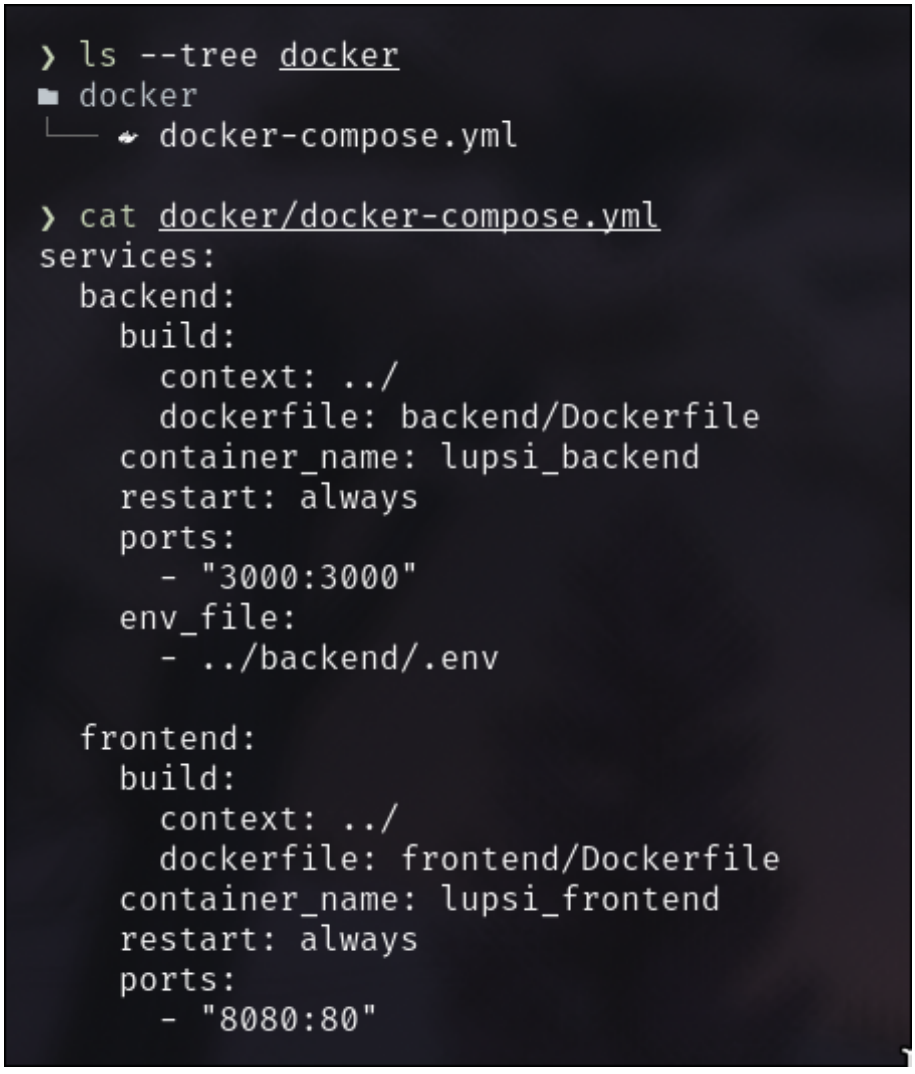


Figura 3: Estructura básica de archivos Docker

En un escenario local de desarrollo, el archivo `docker-compose.yml` puede coordinar el arranque conjunto de frontend, backend y servicios complementarios. Cuando sea necesario, la base de datos o servicios externos podrán mantenerse fuera del contenedor, siempre que el equipo disponga de acceso a las credenciales correspondientes.

8.3. Lineamientos de contenedorización

Para este proyecto se recomienda:

- construir una imagen separada para el backend NestJS;
- construir otra imagen para el frontend Angular cuando sea necesario probar despliegue local;
- usar volúmenes para reflejar cambios de desarrollo sin reconstruir contenedores en cada ajuste;
- declarar variables de entorno mediante archivos `.env` no versionados;
- conservar puertos bien definidos para evitar colisiones entre servicios.

8.4. Buenas prácticas de uso

- No incluir secretos, tokens ni contraseñas dentro de las imágenes.
- Mantener archivos `Dockerfile` limpios y documentados.
- Versionar únicamente la configuración necesaria, no los artefactos generados.
- Priorizar imágenes ligeras para reducir tiempos de construcción.
- Verificar cada servicio antes de integrarlo al flujo común del equipo.

9. Flujo de trabajo local del equipo

Para asegurar una colaboración ordenada, el equipo aplicará el siguiente flujo general:

1. Clonar el repositorio remoto en la estación de trabajo local.
2. Crear una rama de trabajo según la funcionalidad asignada.
3. Desarrollar y probar los cambios en un entorno Docker local.
4. Registrar los avances con commits descriptivos.
5. Subir la rama al repositorio remoto para revisión.

6. Integrar el cambio a la rama de desarrollo una vez validado.

Este esquema permite que el trabajo diario se mantenga trazable y reduce la probabilidad de sobrescribir cambios realizados por otro integrante. Además, facilita la revisión por parte del responsable técnico o del docente patrocinador cuando se requiera evidencia del avance.

10. Estructura sugerida de archivos auxiliares

Además de los directorios principales del sistema, se recomienda mantener archivos de soporte para documentación y operación local:

- `README.md`: descripción general del proyecto y pasos de ejecución.
- `.gitignore`: exclusión de archivos temporales, dependencias y secretos.
- `.env.example`: plantilla de variables de entorno sin valores sensibles.
- `docker-compose.yml`: coordinación de contenedores locales.
- `Dockerfile`: definición de la construcción de imágenes.

11. Criterios de validación del entorno

Antes de iniciar el desarrollo funcional, el equipo debe verificar que la configuración local cumpla con los siguientes criterios:

- Git permite crear, fusionar y sincronizar ramas correctamente.
- Docker levanta los servicios sin errores de compatibilidad.
- El frontend y el backend pueden ejecutarse de forma aislada y conjunta.
- Las variables de entorno se cargan sin exponer credenciales.
- El repositorio conserva un historial claro y ordenado de cambios.

12. Riesgos asociados a la configuración local

Aunque el entorno propuesto es relativamente simple, existen riesgos técnicos que deben controlarse desde el inicio:

Cuadro 1: Riesgos asociados al entorno local

Riesgo	Impacto	Mitigación
Versiones diferentes de herramientas entre integrantes	Alto	Estandarizar versiones mínimas de Git, Node.js y Docker; documentar la configuración inicial.
Pérdida de cambios por trabajo no sincronizado	Medio	Aplicar commits frecuentes y sincronización diaria con el repositorio remoto.
Exposición accidental de credenciales	Alto	Usar archivos `.env` locales y plantillas `.env.example` sin información sensible.
Conflictos entre ramas por cambios simultáneos	Medio	Definir ramas por funcionalidad y realizar revisiones antes de fusionar.

13. Conclusiones

La configuración local de Docker y Git constituye una base esencial para el desarrollo ordenado del sistema LUPSI. Git asegura trazabilidad, colaboración y control de versiones, mientras que Docker aporta consistencia en la ejecución de los servicios y reduce problemas derivados de diferencias entre equipos. En conjunto, ambas herramientas permiten establecer un entorno de trabajo formal, reproducible y acorde con la naturaleza académica y técnica del proyecto.

La adopción de esta configuración desde el inicio fortalece la disciplina de desarrollo del equipo, facilita la revisión de avances y contribuye a que el producto final mantenga estabilidad durante todo el ciclo de construcción.

14. Firmas de Responsabilidad

Rol	Nombre / Representante	Firma (QR)	Fecha
Gestor del Proyecto	Angel Ayuquina		25/03/2026
Desarrollador Backend	Sebastián Ortiz		25/03/2026
Desarrollador Fullstack	Daniel Luisa		25/03/2026
Desarrollador Frontend	Alex Huachi		25/03/2026

Referencias

[1] Git SCM, *Git Documentation*. [En línea]. Disponible en: https://git-scm.com/docs. [Accedido: 24-mar-2026].

[2] Docker, *Docker Documentation*. [En línea]. Disponible en: https://docs.docker.com/. [Accedido: 24-mar-2026].